

IS105 - ENTERPRISE ARCHITECTURE

MR. NIL CARLO M. DELA
CRUZ INSTRUCTOR



UNIFIED MODELING LANGUAGE (UML) DIAGRAMS



AT THE END OF THIS LESSON, YOU SHOULD BE ABLE TO:

- Familiarize with the UML Diagrams.
- Be conscious how important UML Diagram is for visualizing and documenting software systems.
- Learn how to create UML Activity Diagram using Enterprise Architect tool.

UML ACTIVITY

DIAGRAM Activity diagrams are a valuable tool in Unified

Modeling Language (UML) for visually representing the flow of control within a system.

- They effectively illustrate the sequential and concurrent steps involved in executing a use case, providing a clear overview of the workflow.

TYPES OF UML

DIAGRAMS
Structure Diagrams - These diagrams focus on the static structure of a system, such as its classes, components, and relationships.

- **Interaction Diagrams** - Interaction diagrams illustrate the interactions between objects in a system, including sequence diagrams and communication diagrams.

TYPES OF UML

DIAGRAMS

Behavior Diagrams - Behavior

diagrams depict the dynamic aspects of a system, such as its behavior over time.

- Activity diagrams fall under this category.

APPLICATIONS OF ACTIVITY

DIAGRAM
Business Process Modeling - They help visualize and understand complex business processes, identifying potential bottlenecks and inefficiencies.

- **Software Development** - Activity diagrams can be used to model the workflow of a software system, aiding in design and implementation.

APPLICATIONS OF ACTIVITY DIAGRAM

- **System Analysis** - They assist in analyzing the dynamic behavior of a system, identifying potential issues and areas for improvement.

FLOWCHART VS ACTIVITY DIAGRAM



Feature	Flowchart	Activity Diagram
Complexity	Simple	Can handle complex processes
Focus	General process representation	Software development, business process modeling
Elements	Basic shapes (rectangles, diamonds)	Swimlanes, object flows, data flow
Decision-making	Simple decisions	Complex decision-making

ACTIVITY DIAGRAM NOTATIONS



INITIAL STATE

- The initial state in an activity diagram represents the starting point of a process or activity.
- It's typically depicted as a filled-in circle. Generally, there's only one initial state per diagram, unless you're dealing with nested activities.



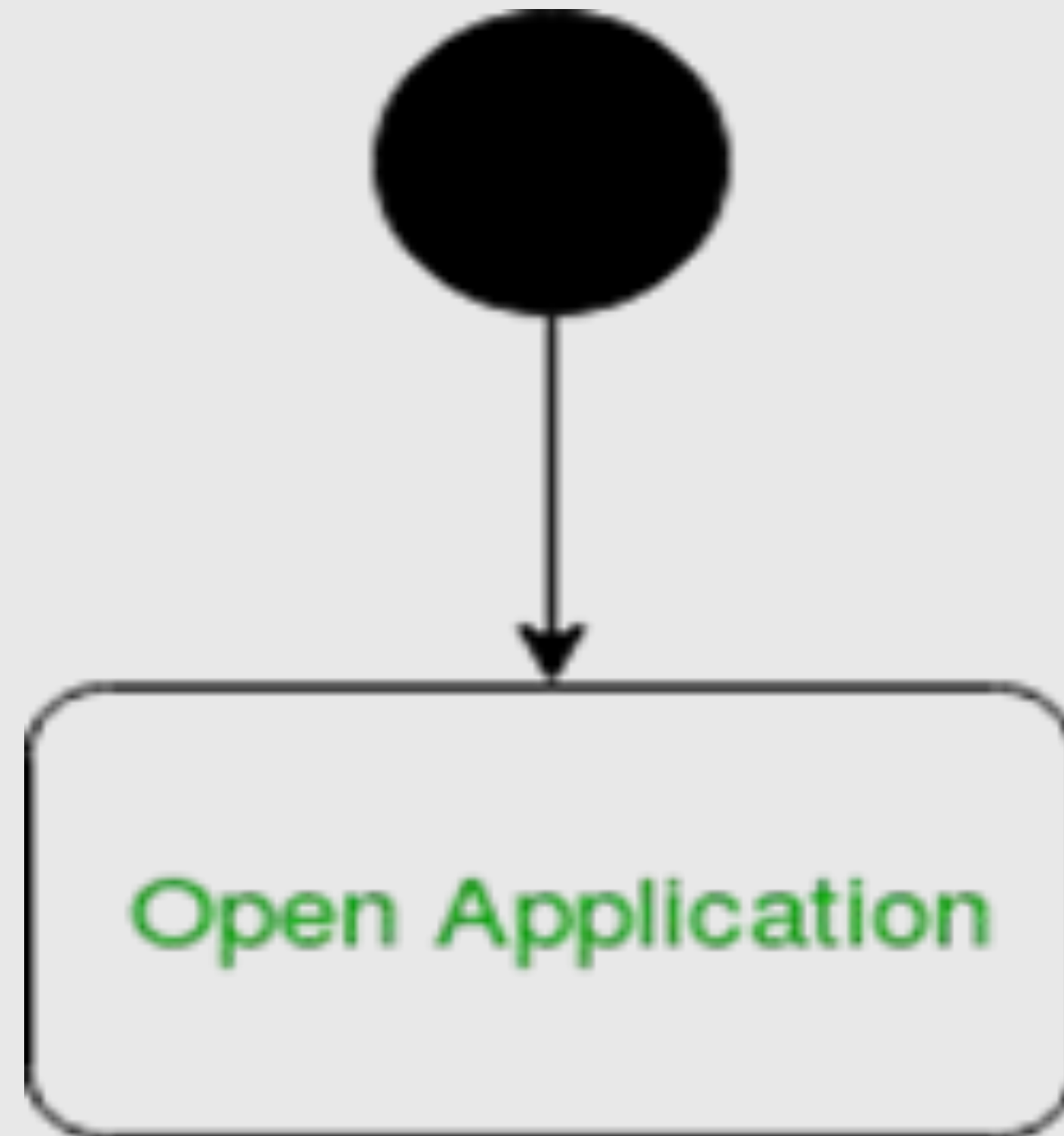
Figure – notation for initial state or start state

ACTION OR ACTIVITY

STATE An activity is a step or action that takes place within a process.

- It's represented as a rounded rectangle in an activity diagram.
- Activities can be anything from opening an app to making a calculation or sending a message.

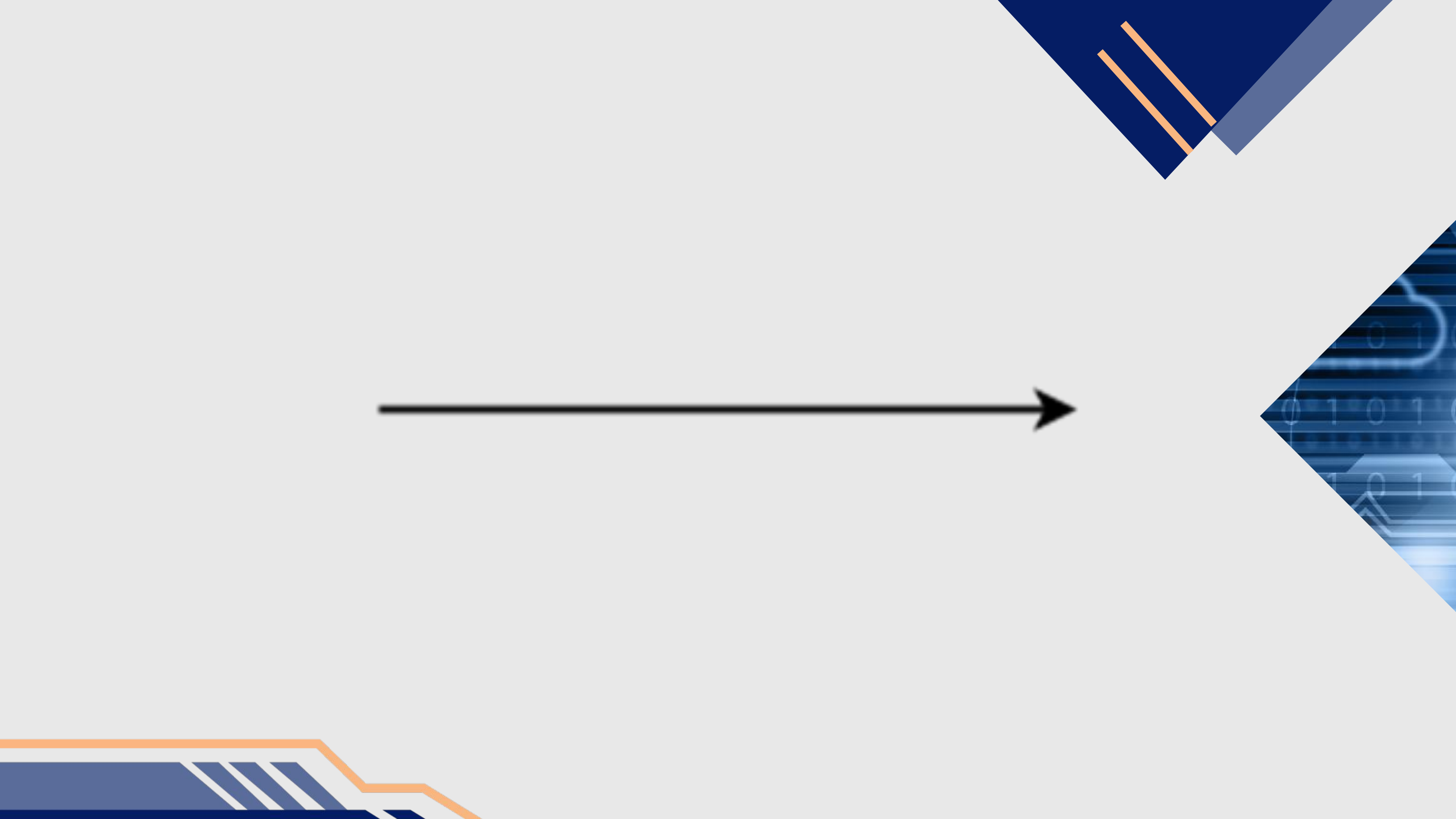
Action or Activity State

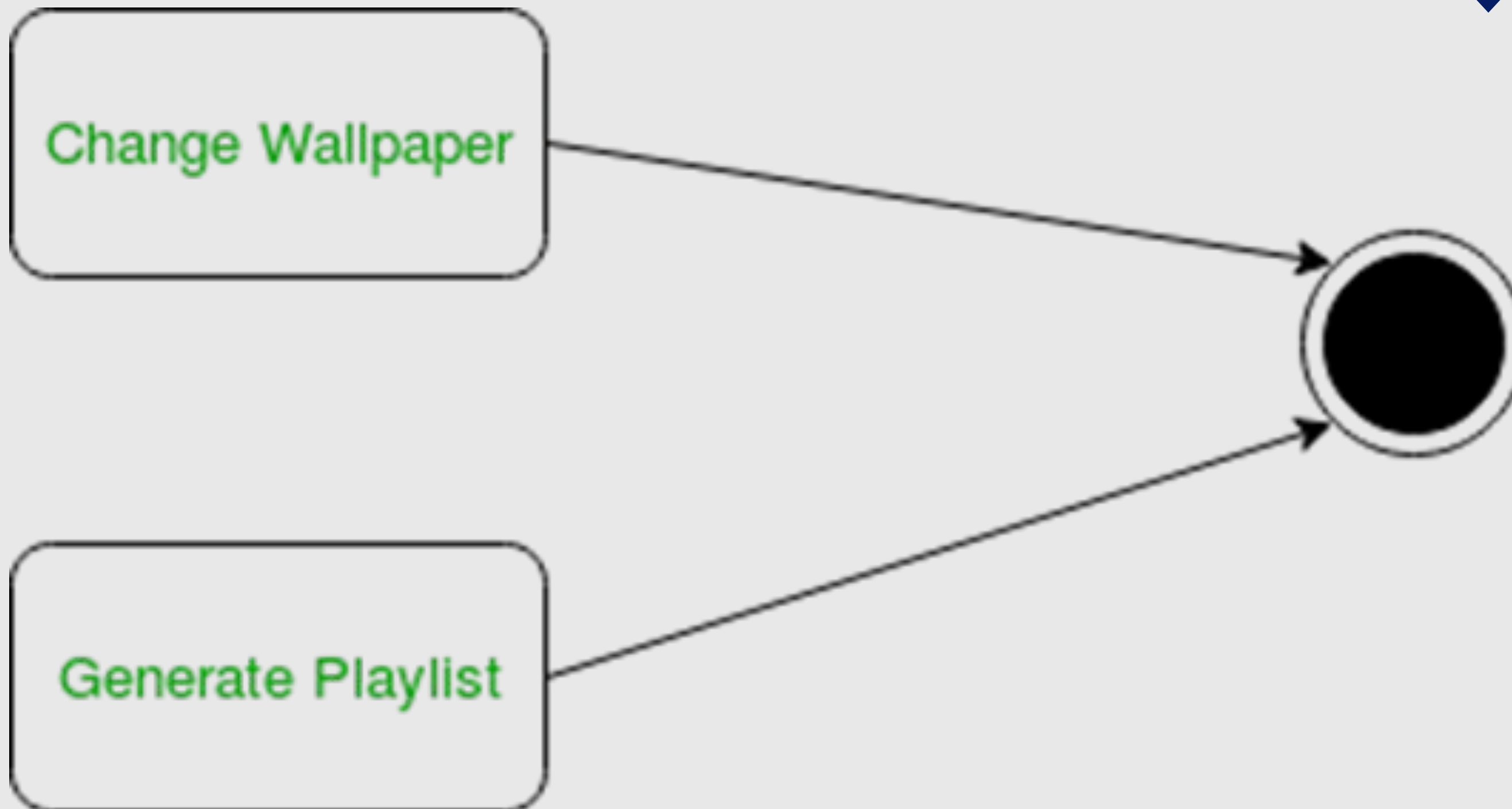


ACTION FLOW OR CONTROL FLOWS

Action flows or Control flows are also referred to as paths and edges.

- They are used to show the transition from one activity state to another.
- An activity state can have multiple incoming and outgoing action flows. We use a line with an arrow head to depict a Control Flow.

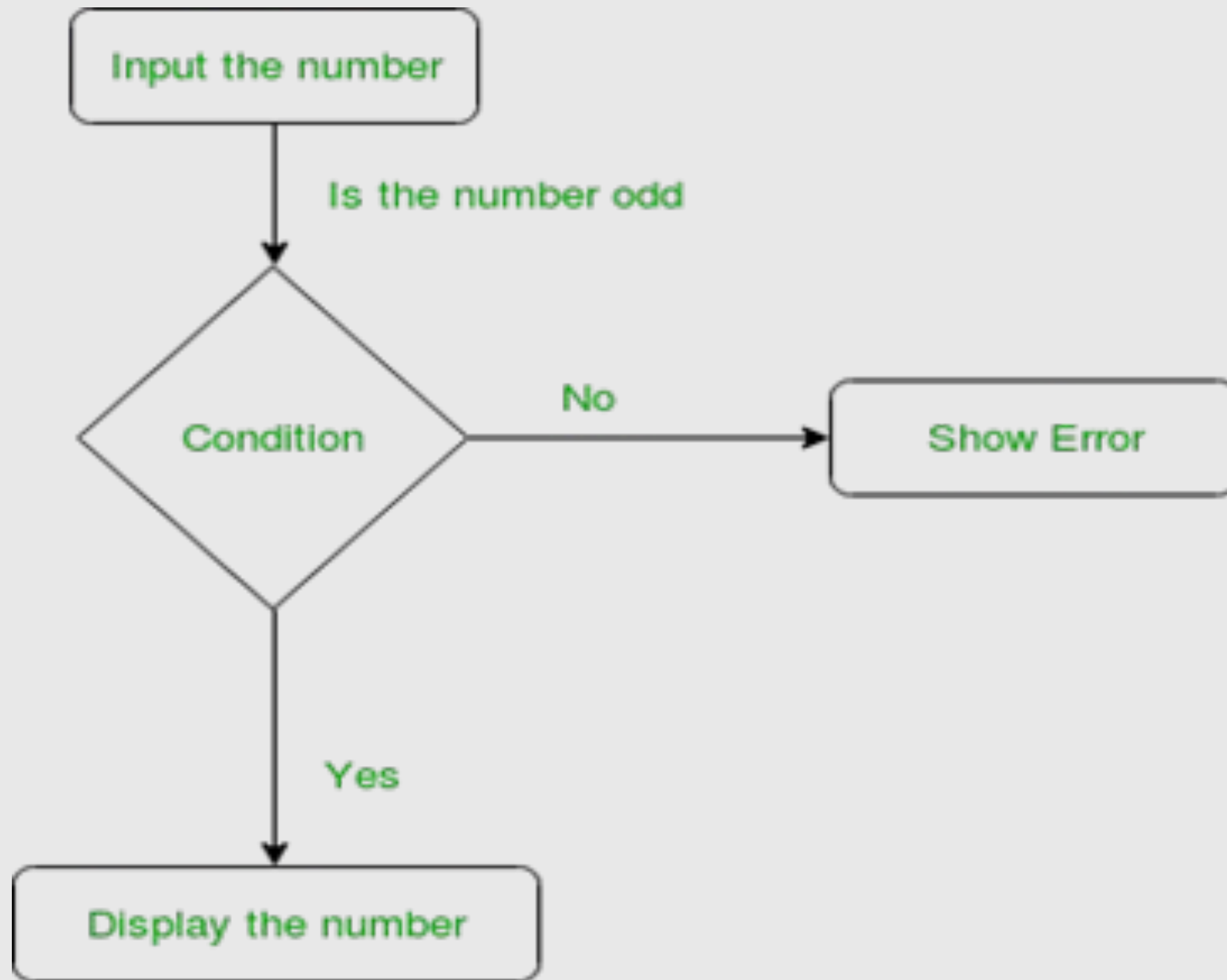




DECISION NODE AND BRANCHING

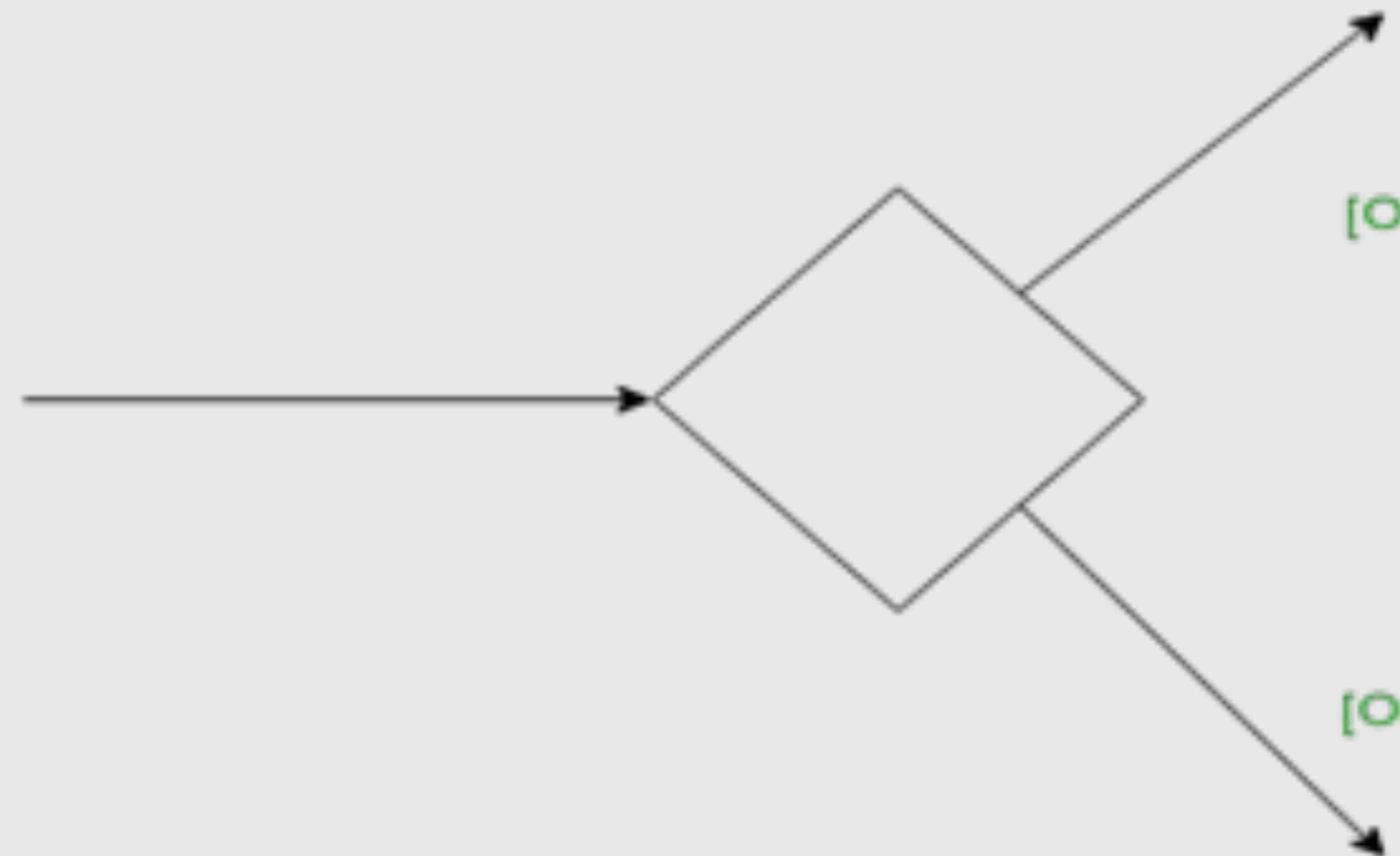
When we need to make a decision before deciding the flow of control, we use the decision node.

- Decision nodes provide flexibility in modeling processes that require conditional branching, making activity diagrams more adaptable to real-world scenarios



GUARDS

- A Guard refers to a statement written next to a decision node on an arrow sometimes within square brackets.
- It is a Boolean expression written near a decision node on an arrow, often enclosed in square brackets. It specifies the condition that must be true for the flow to follow that particular outgoing edge.



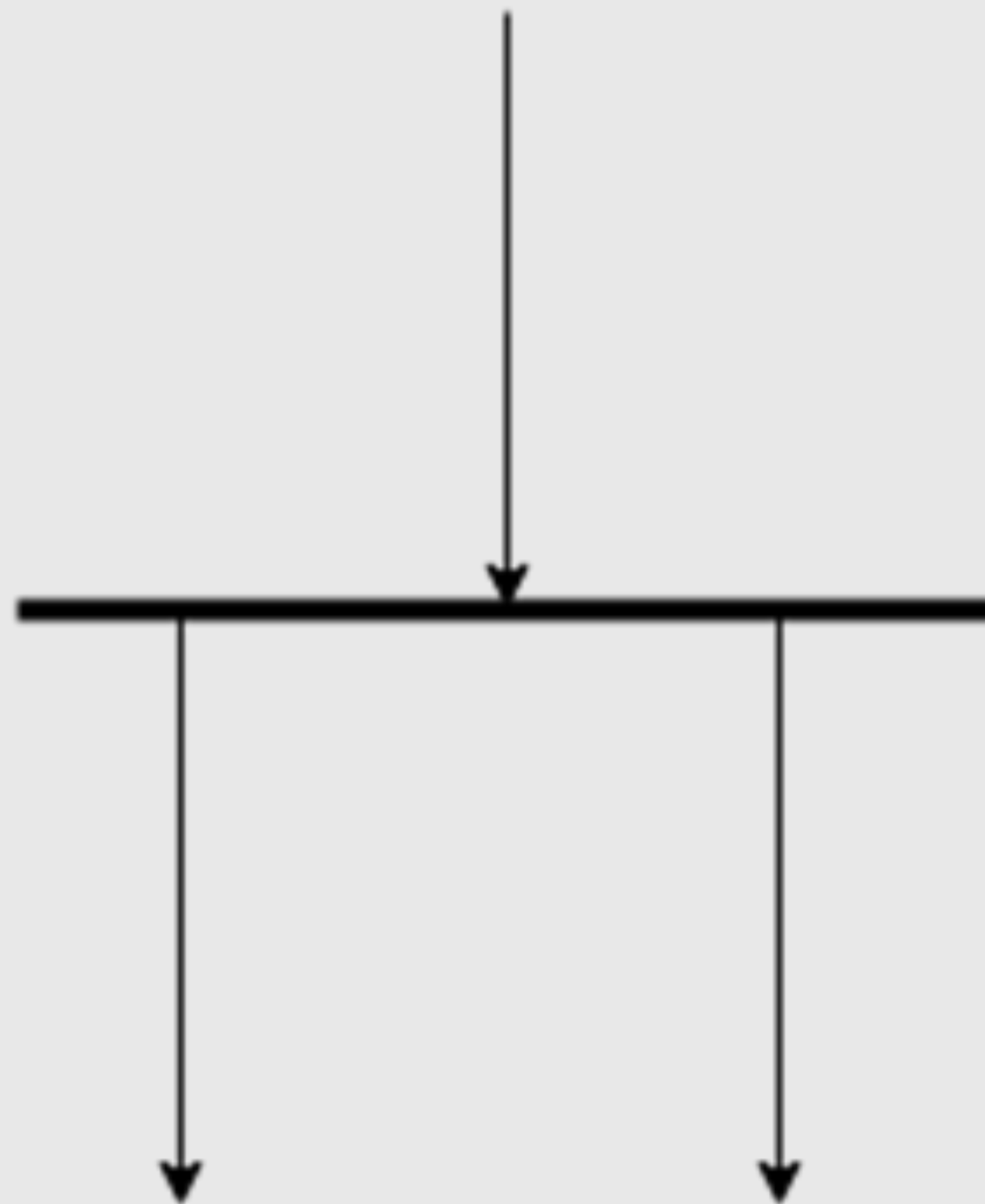
```
graph LR; Start(( )) --> Decision{ }; Decision --> Accepted([Order accepted]); Decision --> Rejected([Order rejected]);
```

[Order accepted]

[Order rejected]

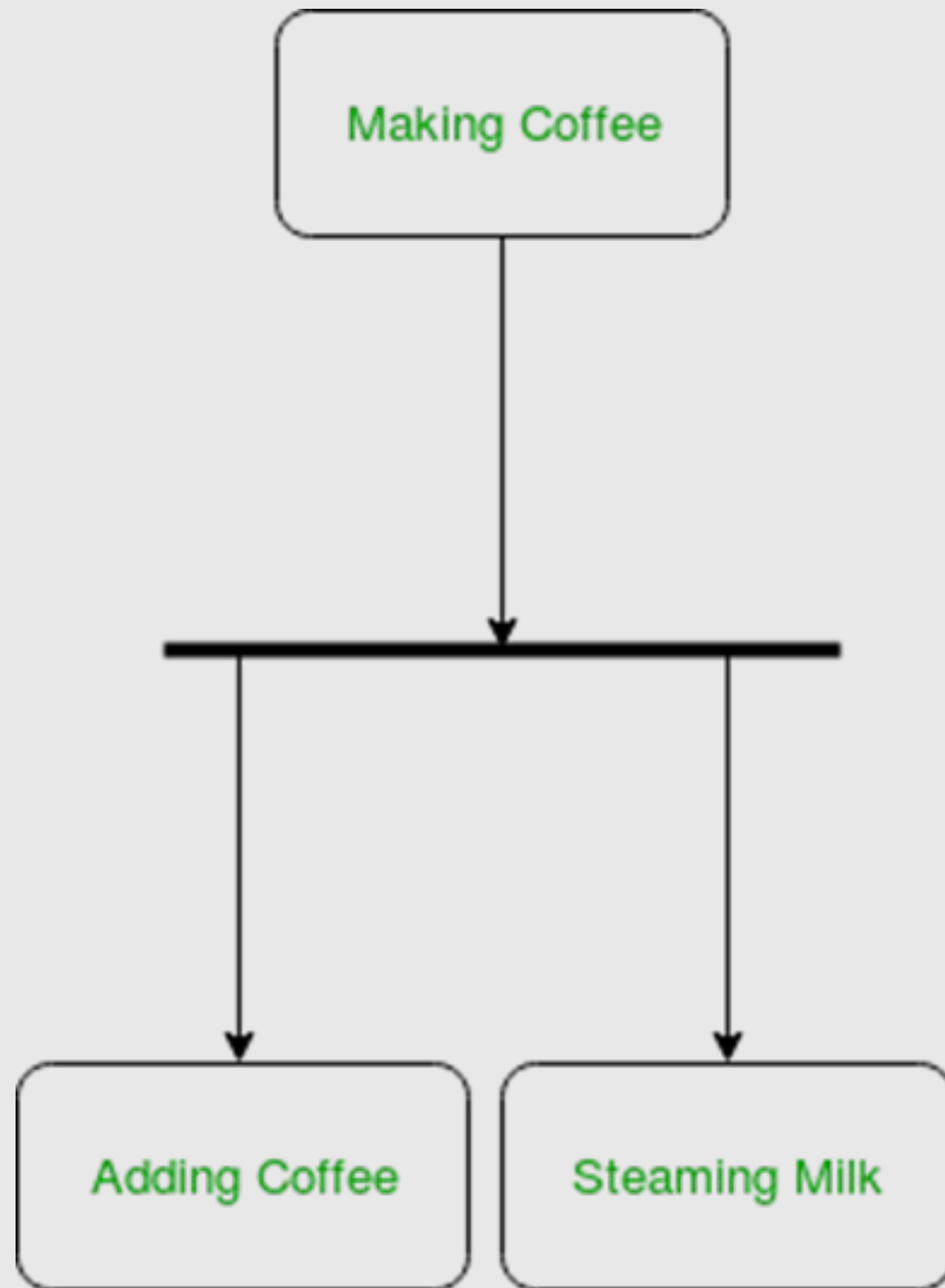
FORK

- Fork nodes are used to support concurrent activities.
- When we use a fork node when both the activities get executed concurrently i.e. no decision is made before splitting the activity into two parts.
- Both parts need to be executed in case of a fork statement.



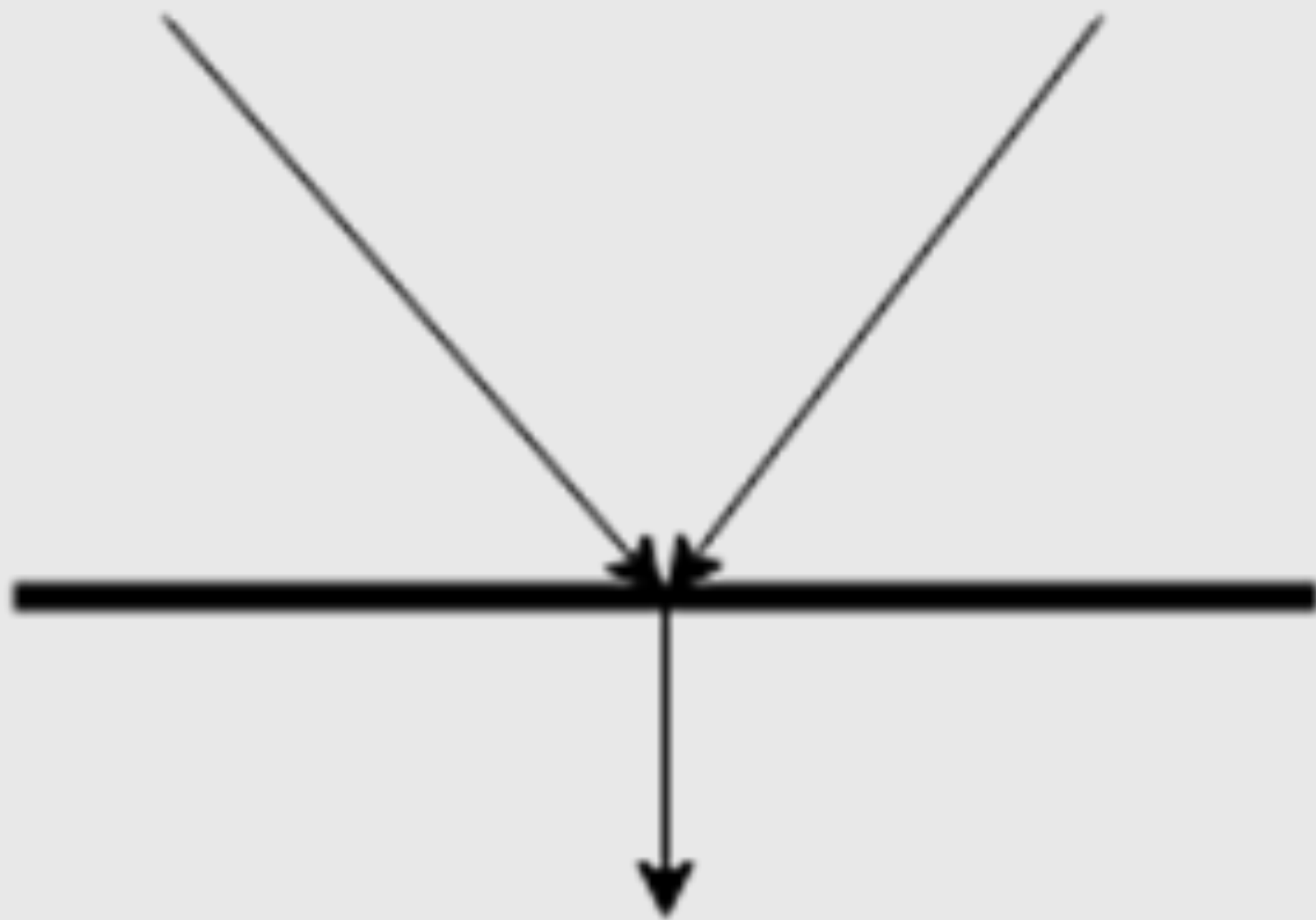
FORK

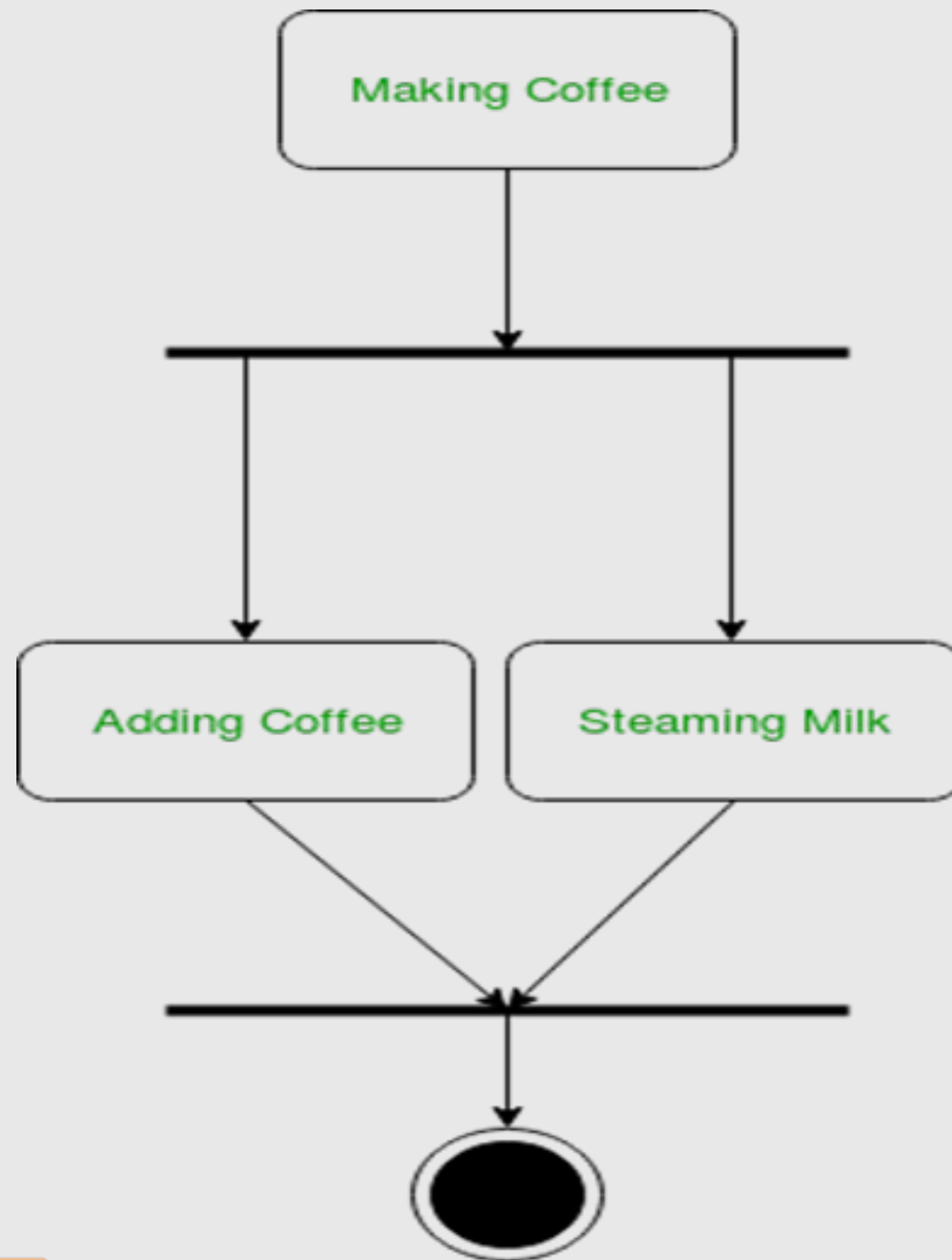
- We use a rounded solid rectangular bar to represent a Fork notation with incoming arrow from the parent activity state and outgoing arrows towards the newly created activities.
- For example, the activity of making coffee can be split into two concurrent activities and



JOIN

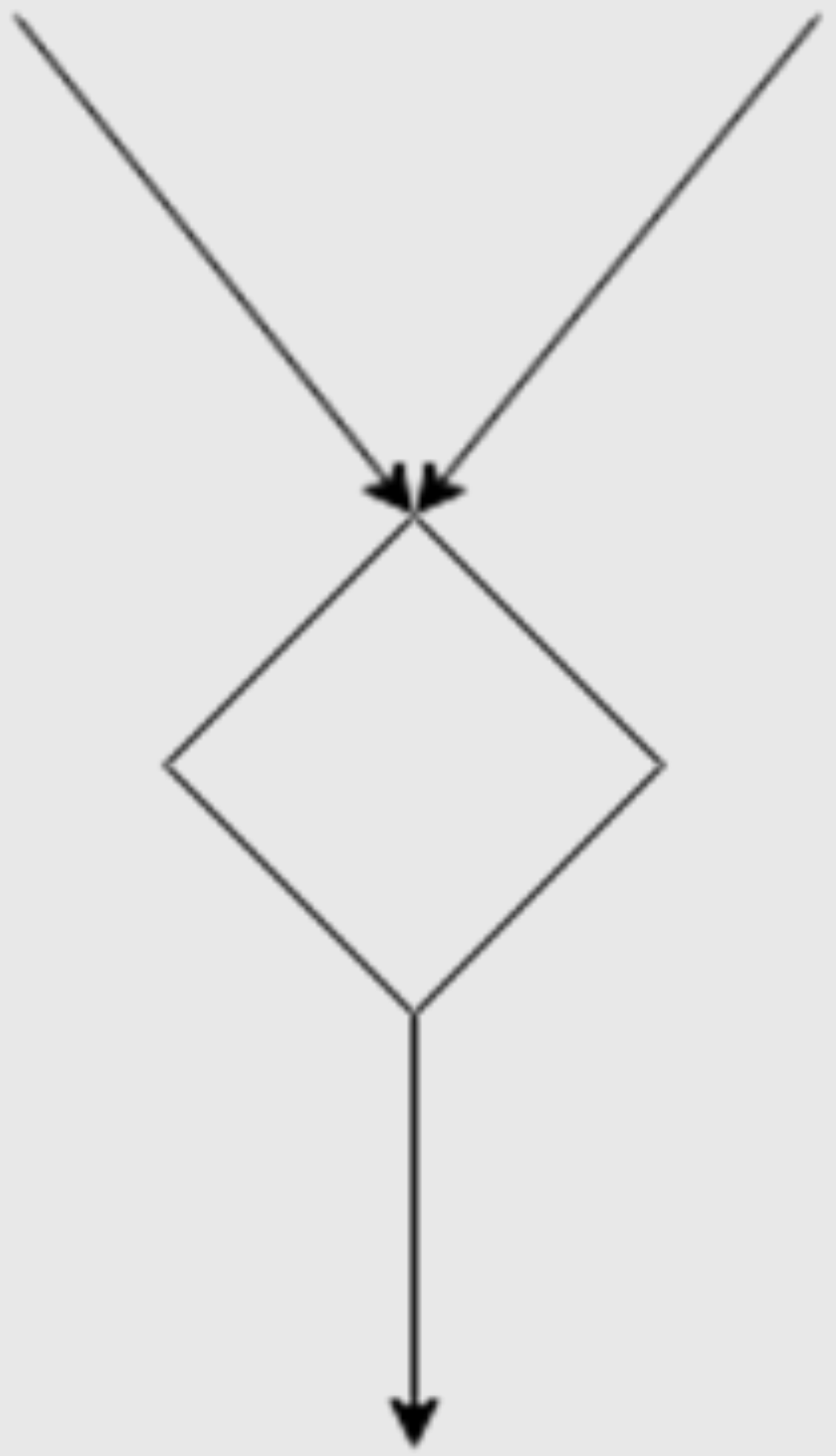
- Join nodes are used to support concurrent activities converging into one.
- For join notations we have two or more incoming edges and one outgoing edge.

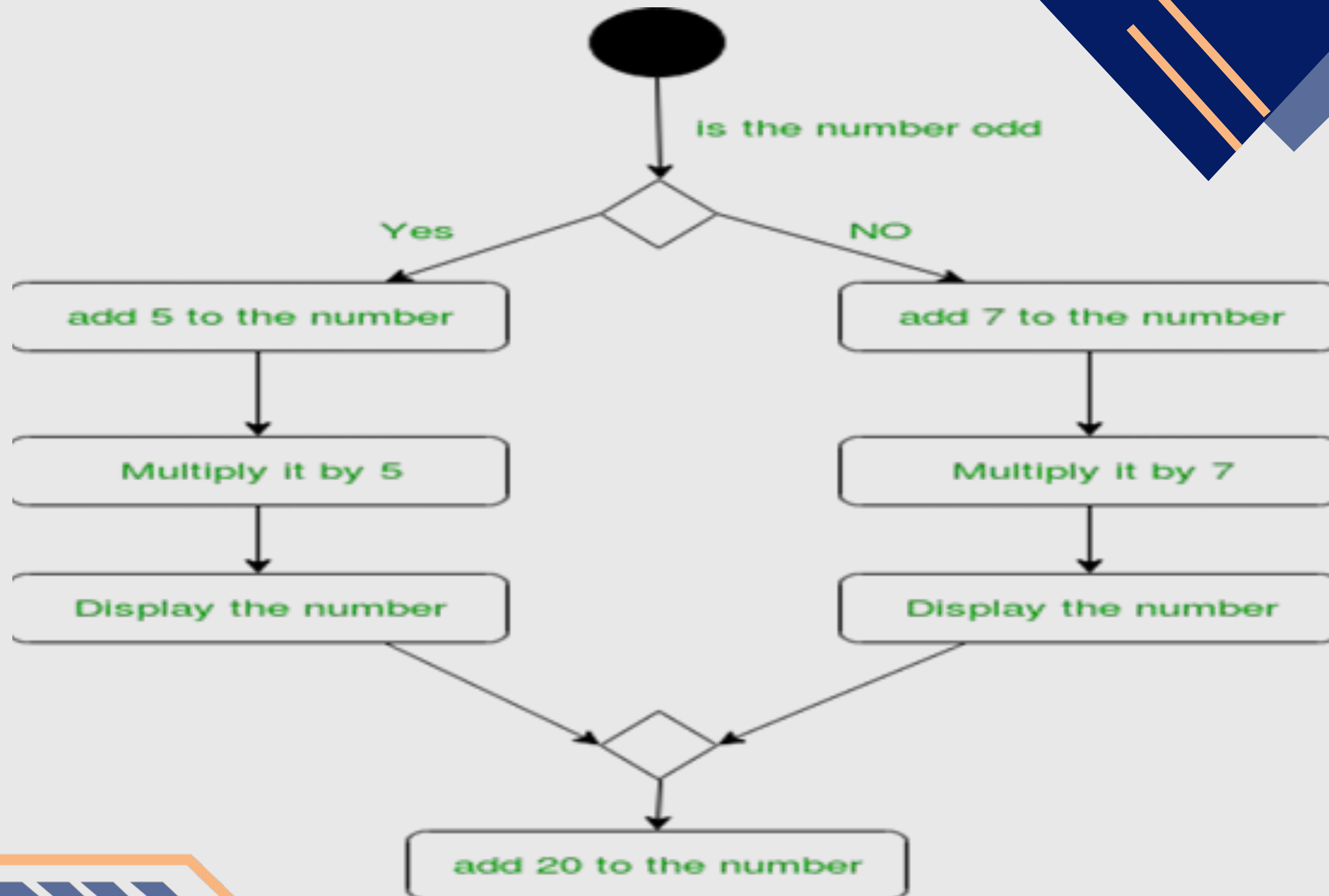




MERGE OR MERGE EVENT

- Scenarios arise when activities which are not being executed concurrently have to be merged.
- We use the merge notation for such scenarios.
- We can merge two or more activities into one if the control proceeds onto the next activity irrespective of the path chosen.





SWIMLANES

- We use swimlanes for grouping related activities in one column.
- Swimlanes group related activities into one column or one row.
- Swimlanes can be vertical and horizontal. Swimlanes are used to add modularity to the activity diagram.

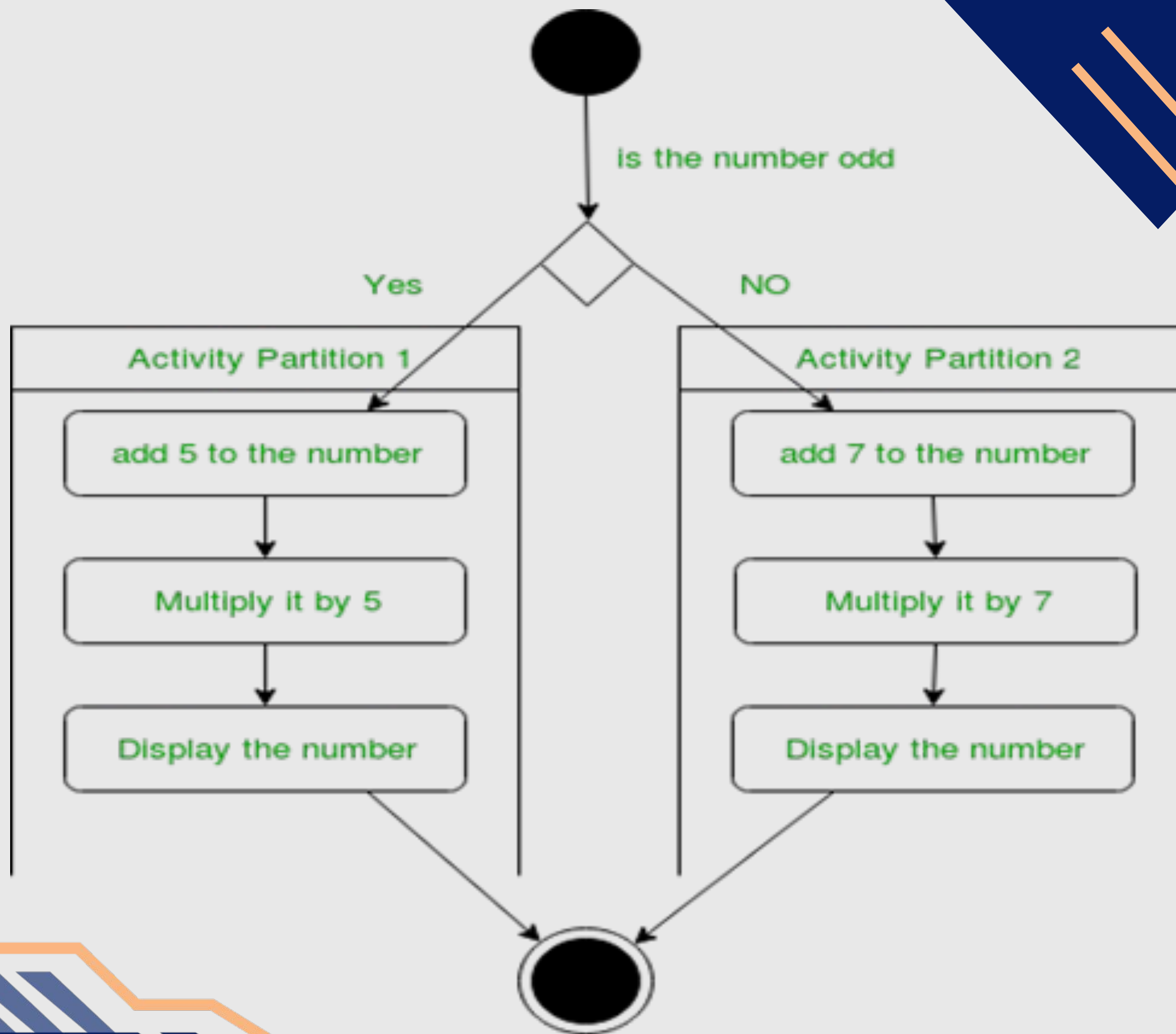
SWIMLANES

- It is not mandatory to use swimlanes. They usually give more clarity to the activity diagram.
- It's similar to creating a function in a program.
- It's not mandatory to do so, but, it is a recommended practice.

Swimlane

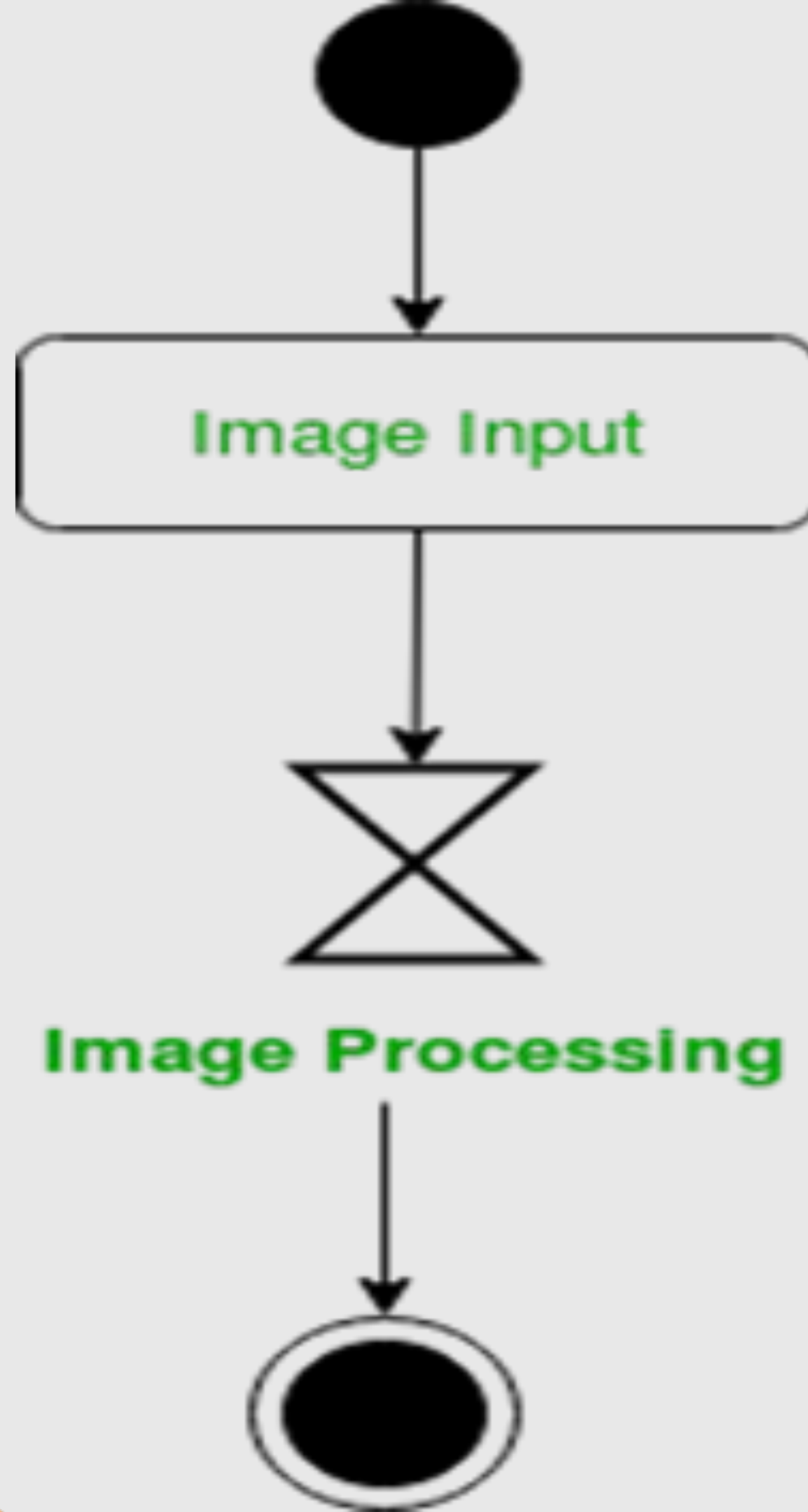
SWIMLANES

- For example, different set of activities are executed based on if the number is odd or even.
- These activities are grouped into a swimlane.



TIME EVENT

- We can have a scenario where an event takes some time to complete.
- We use an hourglass to represent a time event.
- For example, let us assume that the processing of an image takes a lot of time. Then it can be represented as shown below.



FINAL STATE OR END STATE

- The state which the system reaches when a particular process or activity ends is known as a Final State or End State.
- We use a filled circle within a circle notation to represent the final state in a state machine diagram.
- A system or a process can have multiple final states.



WEEK 5

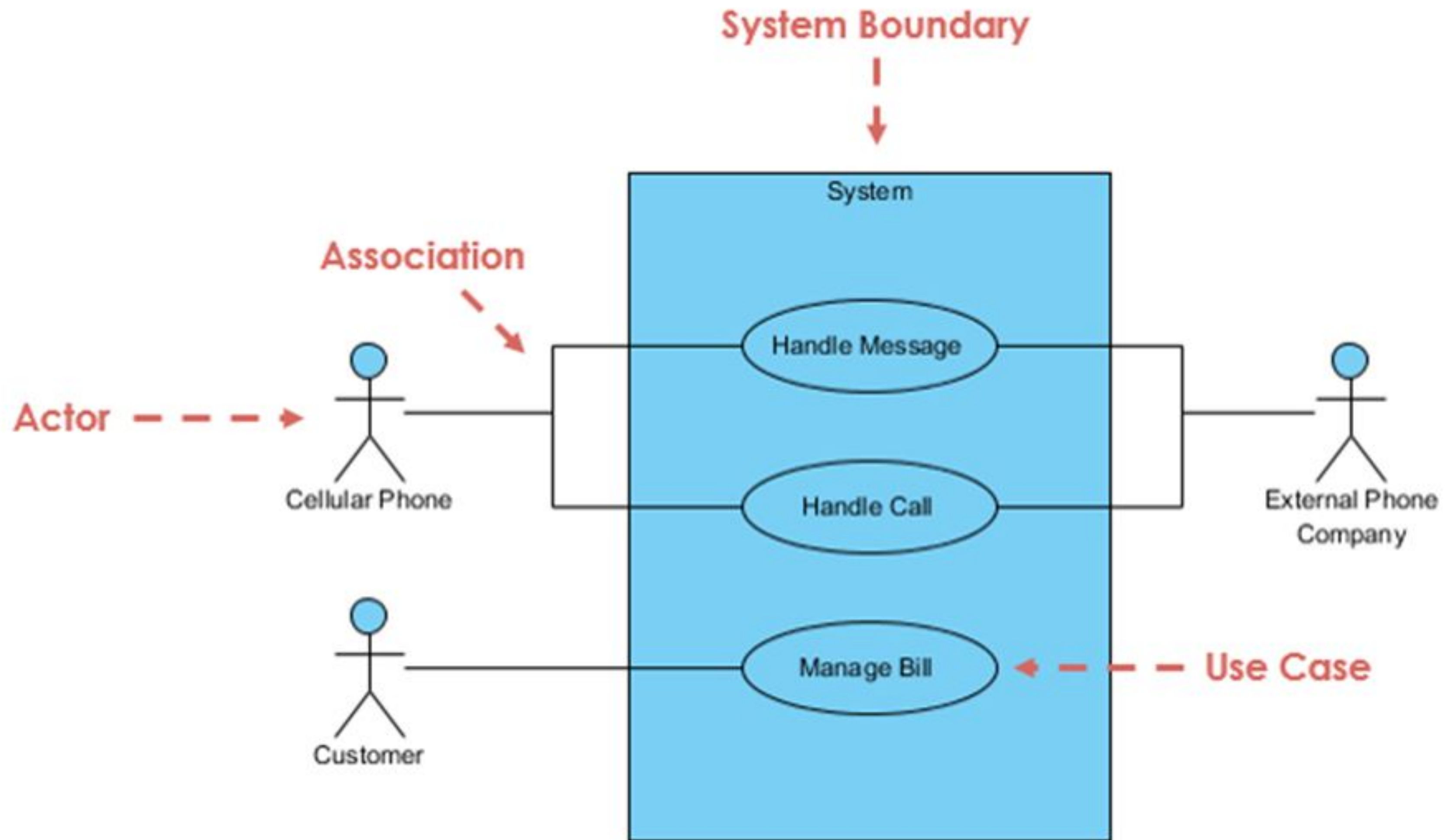


USE CASE DIAGRAM



USE CASE

- Use cases describe the interactions between users and a system to achieve specific goals.
- Activity diagrams are visual representations of the steps involved in executing these use cases.
- Together, they provide a comprehensive understanding of a system's functionality.
- Use cases define the what, while activity



IMPORTANCE OF USE CASE

- Use cases are a valuable tool for understanding and designing systems.
- They outline the interactions between users and a system to achieve specific goals.
- By using use cases, you can clearly define the system's functionalities, understand the roles of different stakeholders, explore user needs, and anticipate potential challenges.

USE-CASE DIAGRAM NOTATIONS



USE-CASE

- Use cases are used to represent high-level functionalities and how the user will handle the system.
- A use case represents a distinct functionality of a system, a component, a package, or a class.
- It is denoted by an oval shape with the name of a use case written inside the oval shape.

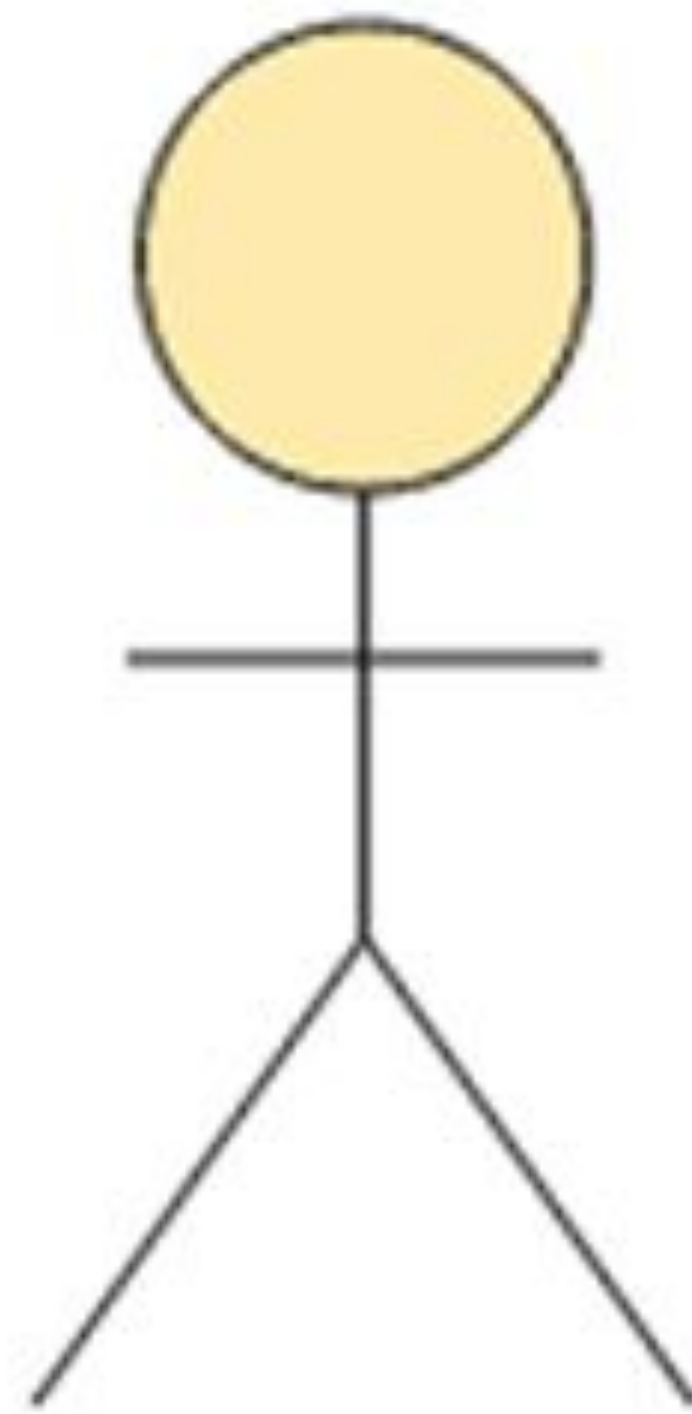


ACTOR

- It is used inside use case diagrams.
- The actor is an entity that interacts with the system.
- A user is the best example of an actor.

ACTOR

- An actor is an entity that initiates the use case from outside the scope of a use case.
- It can be any element that can trigger an interaction with the use case.
- One actor can be associated with multiple use cases in the system.



Actor-name

ASSOCIATIONS

- It is a lines connecting actors to use cases, indicating which actors can participate in which use cases.
- Associations make it clear which actors are involved in each use case, improving understanding of the system's functionality.

ASSOCIATION LINK



HOW TO DRAW A USE-CASE DIAGRAM?



INSTRUCTIONS

- To create a use case diagram, start by analyzing the system's functions and identifying the entities that will interact with it.
- Convert the functions into use cases and define the relationships between these use cases and the actors.

INSTRUCTIONS

- Remember to create detailed descriptions for each use case and consider relationships between them.
- Use case modeling is often an iterative process, so be prepared to refine your diagram as you gain more understanding of the system.

INSTRUCTIONS

- The name of an actor or a use case must be meaningful and relevant to the system.
- Interaction of an actor with the use case must be defined clearly and in an understandable way.
- Annotations must be used wherever they are required.
- If a use case or an actor has multiple relationships, then only significant interactions must be displayed.

TIPS

- A use case diagram should be as simple as possible.
- A use case diagram should be complete.
- A use case diagram should represent all interactions with the use case.

TIPS

- If there are too many use cases or actors, then only the essential use cases should be represented.
- A use case diagram should describe at least a single module of a system.
- If the use case diagram is large, then it should be generalized.

STRUCTURING USE CASE DIAGRAM WITH

RELATIONSHIPS

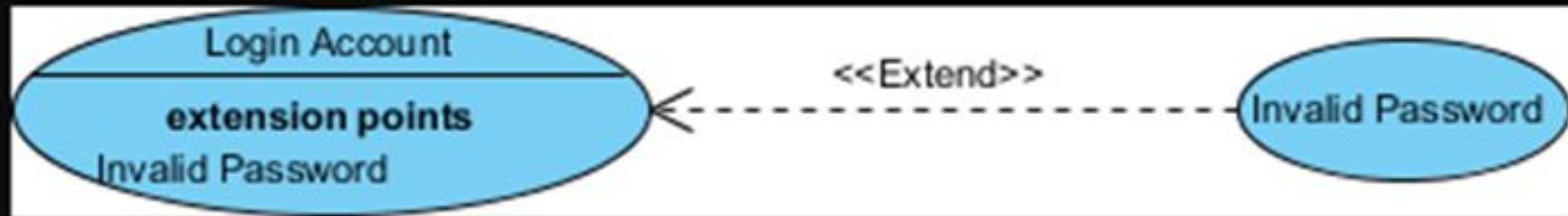
- Defining the relationship between two use cases is the decision of the software analysts of the use case diagram.
- A relationship between two use cases is basically modeling the dependency between the two use cases.

EXTENDS

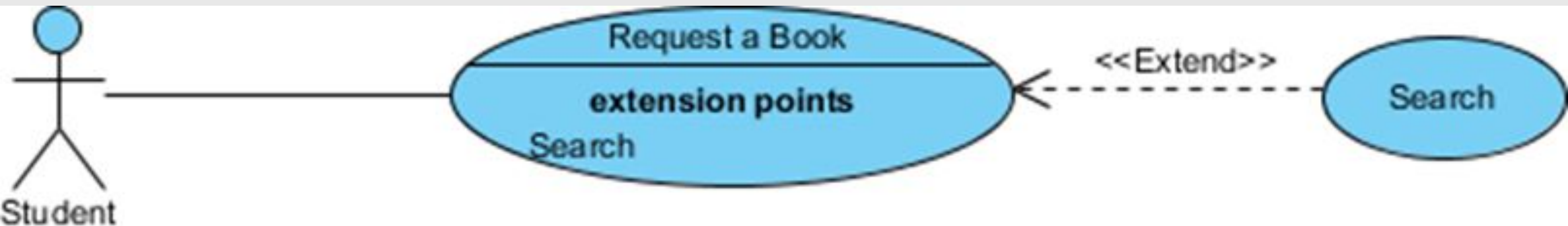
- Extending use cases allow you to add optional or conditional behavior to existing use cases.
- You can use a dotted arrow with the stereotype "<<extends>>" to show this relationship.

EXTENDS

- For example, an "Invalid Password" use case might extend the "Login Account" use case, indicating that if a user enters an incorrect password, additional actions like displaying an error message or locking the account can be performed.



EXTEND RELATIONSHIP



INCLUDE

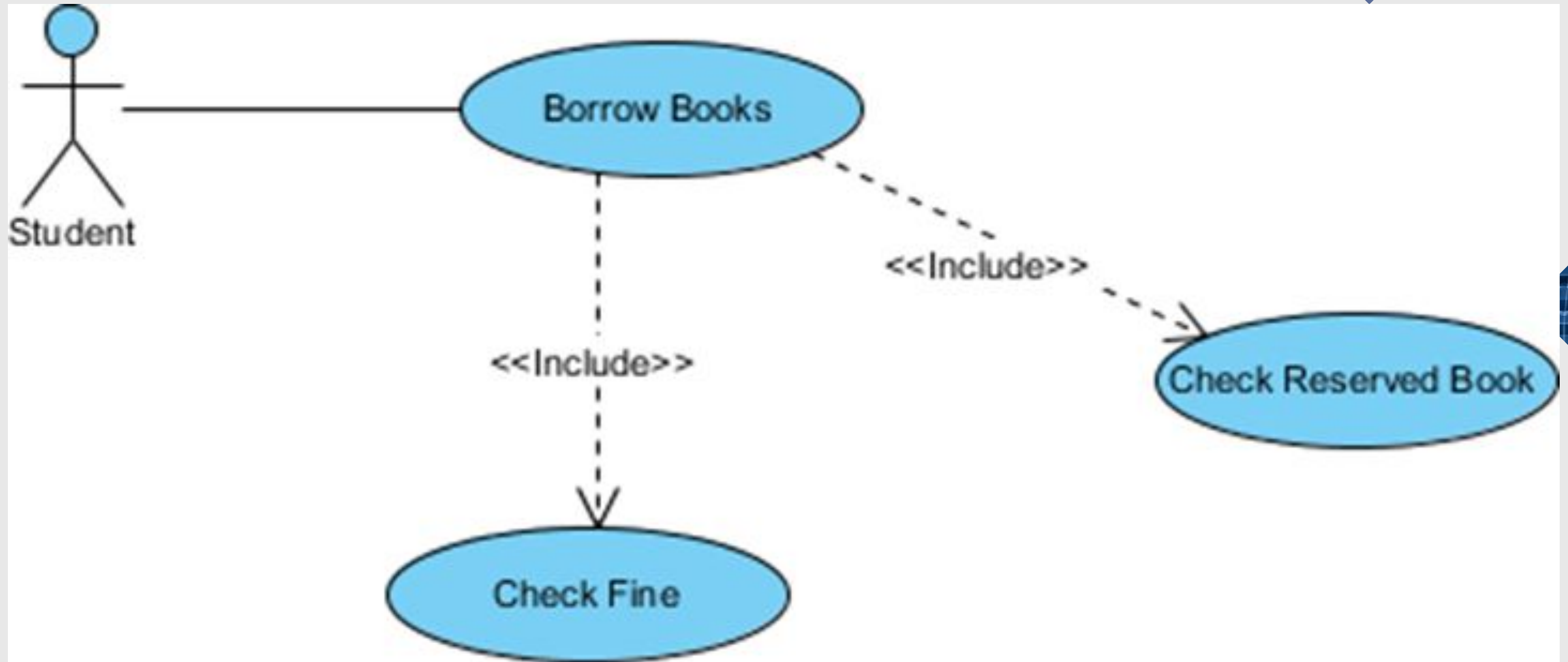
- When a use case is depicted as using the functionality of another use case, the relationship between the use cases is named as include or uses relationship.
- Include relationships in use case diagrams show when a use case is a necessary part of another use case.

INCLUDE

- For example, a "Place Order" use case might include the "Add Items to Cart" use case, meaning that customers must add items to their cart before placing an order.
- This relationship is shown with a dotted arrow from the base use case to the included use case.



INCLUDE RELATIONSHIP



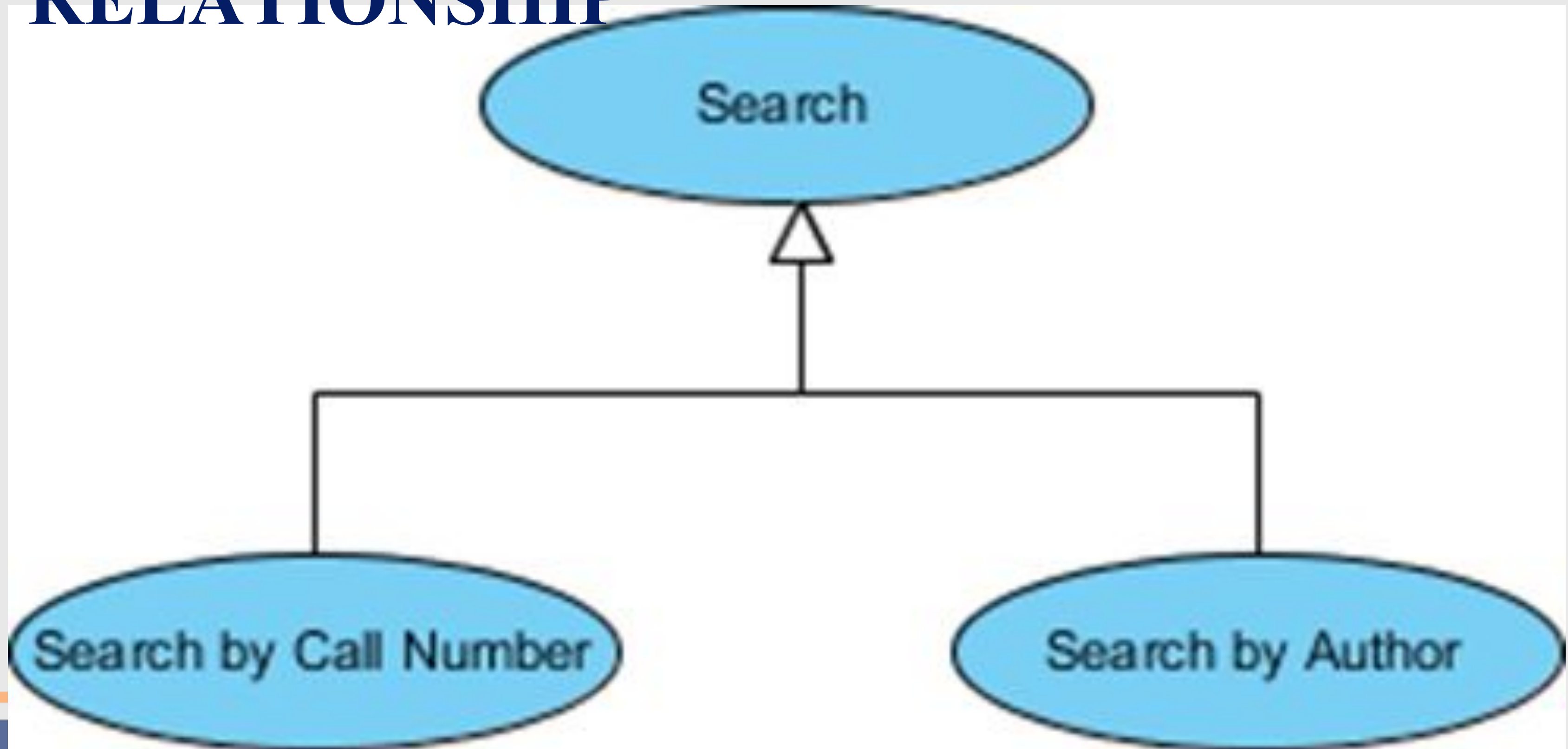
GENERALIZATION

- Generalization relationships in use case diagrams show when a use case is a specialized version of another use case.
- For example, "Make Credit Card Payment" and "Make Debit Card Payment" could be generalizations of the "Make Payment" use case.

GENERALIZATION

- This relationship is shown with an arrow pointing from the child use case (e.g., "Make Credit Card Payment") to the parent use case (e.g., "Make Payment").

GENERALIZATION RELATIONSHIP

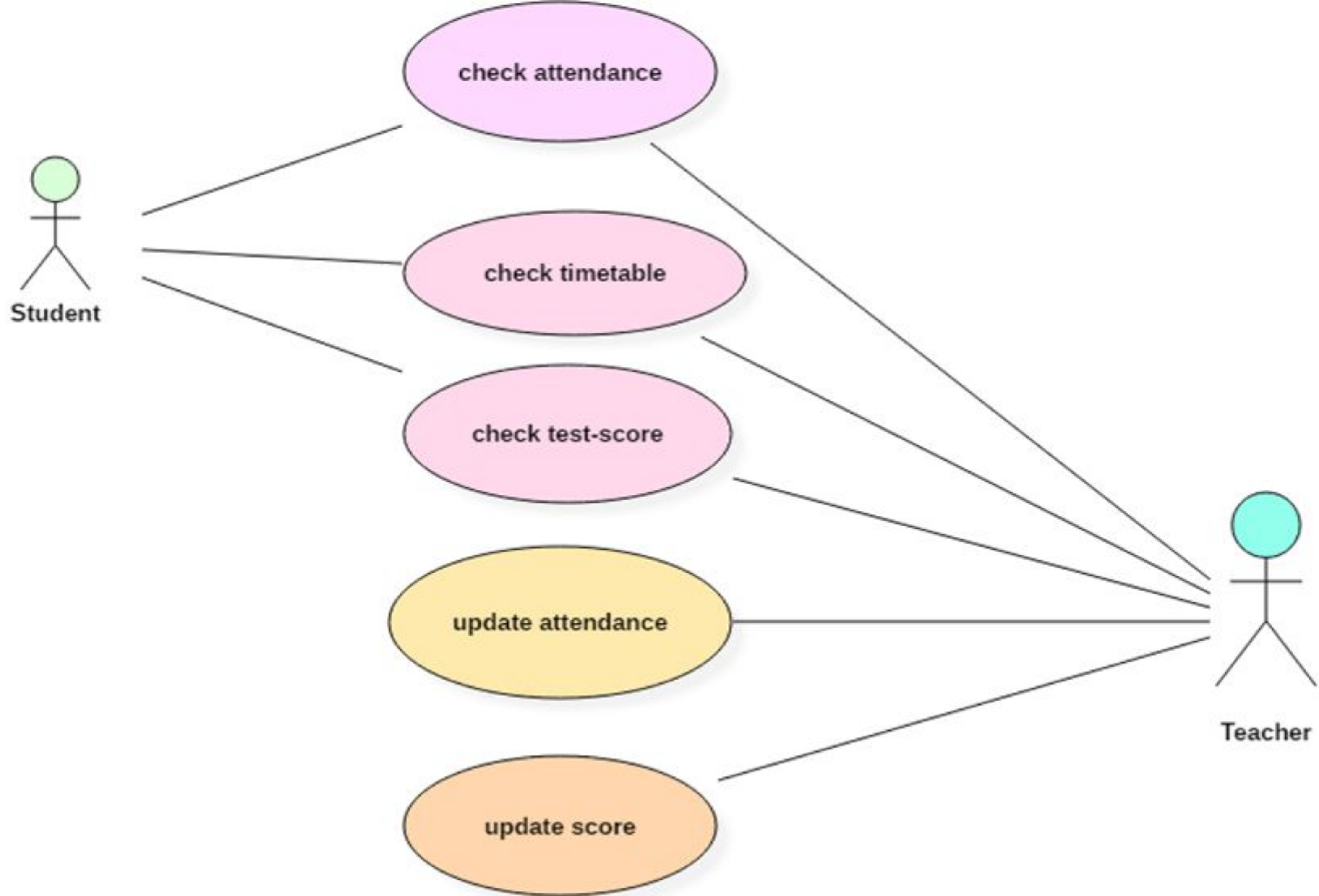


USE-CASE DIAGRAM EXAMPLES



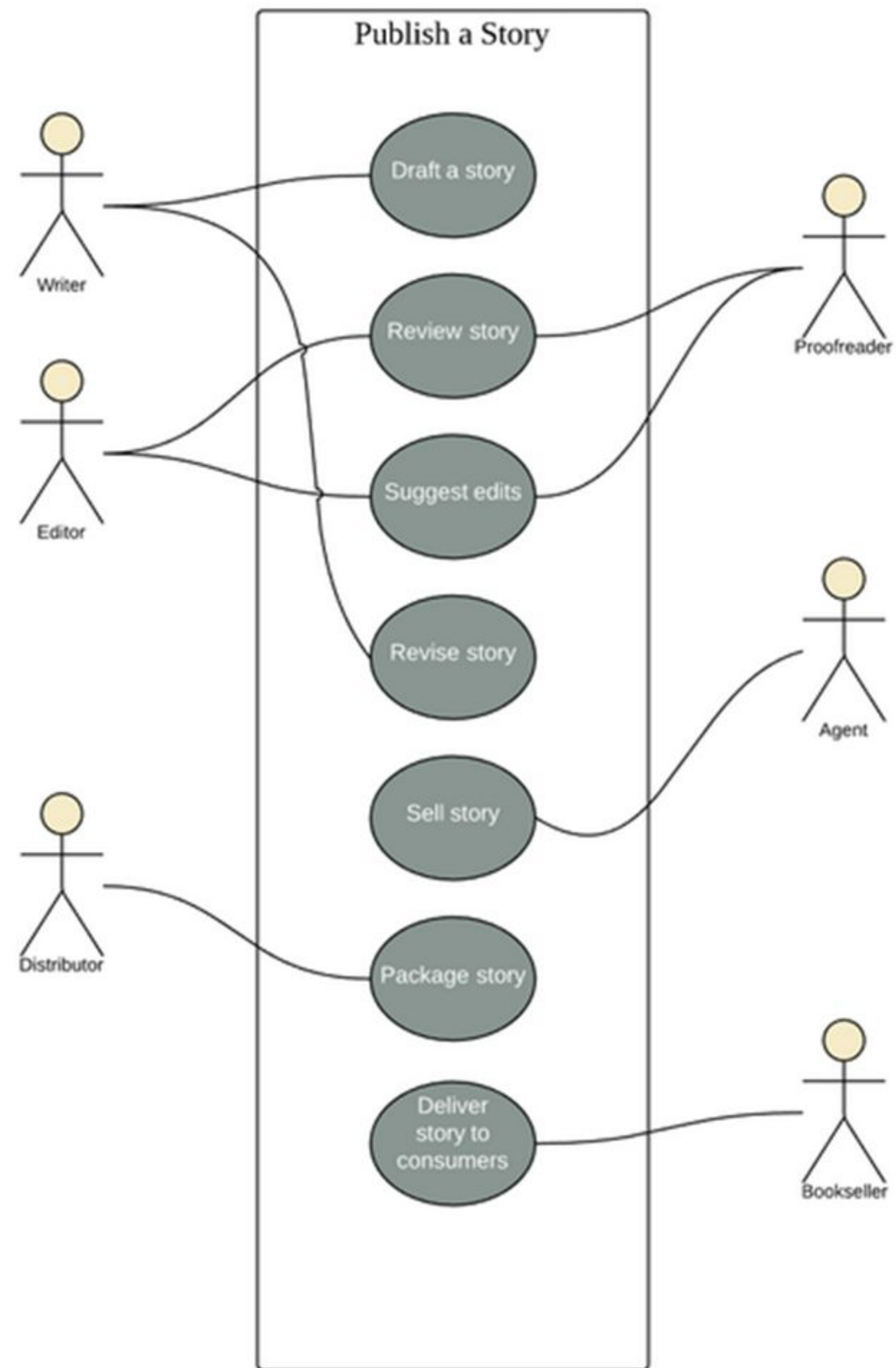
STUDENT MANAGEMENT SYSTEM





BOOK PUBLISHING USE CASE DIAGRAM

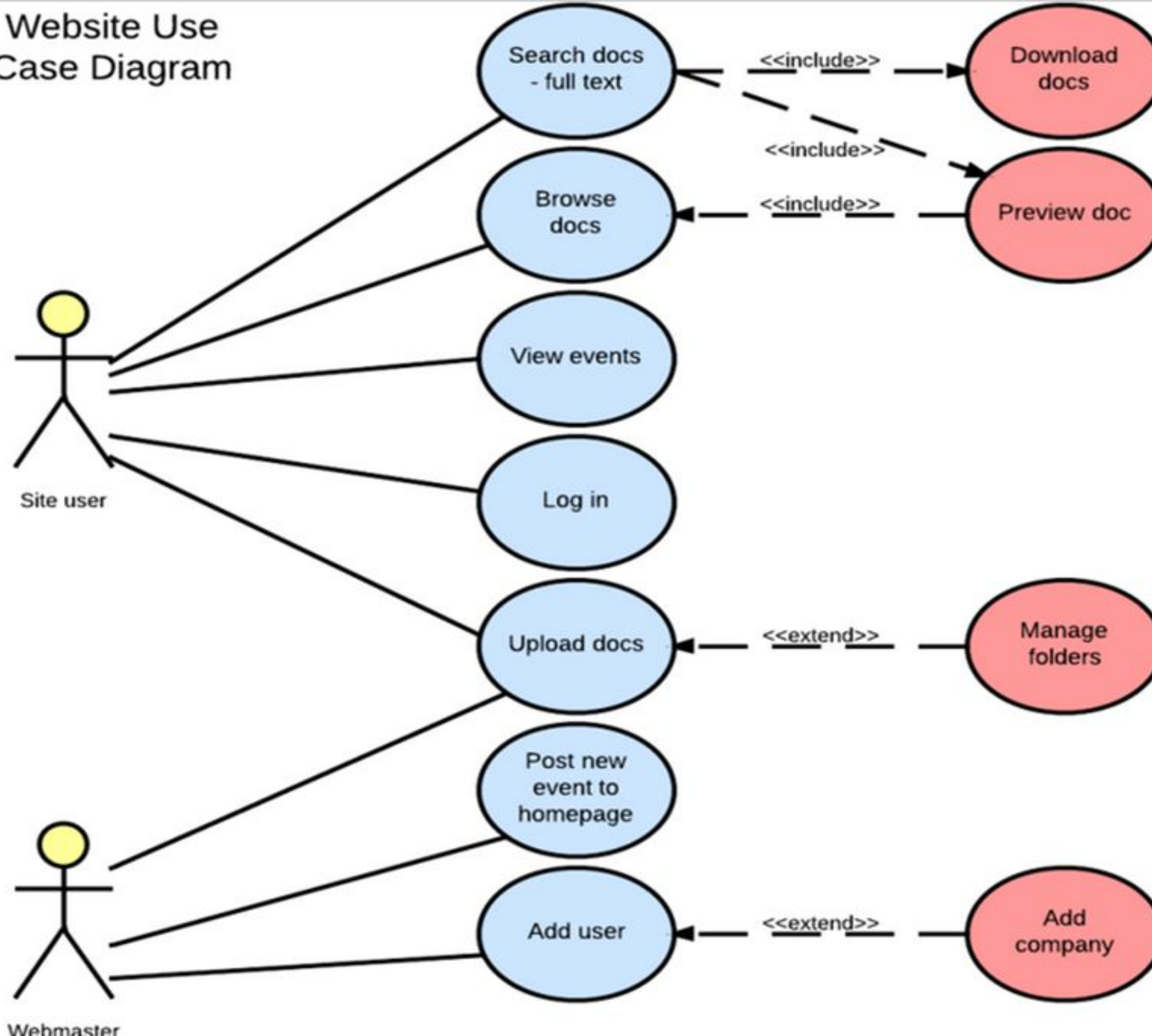




WEBSITE USE CASE DIAGRAM



Website Use Case Diagram



THE END